

Betriebssysteme Übungsblatt 2

1. Einleitung

Das „Santa Claus Problem“ von John Trono aus dem Jahr 1994 ist ein populäres Problem zur Demonstration der Prozesssynchronisation. Es wird ein festliches Szenario, mit dem Weihnachtsmann als zentralem Prozess simuliert, der von zwei unterschiedlichen Prozessgruppen aus seinem wohlverdienten Schlaf geweckt wird. Dabei gelten strikte Prioritätsregeln und Kapazitätsgrenzen:

- Der **Weihnachtsmann** darf nur geweckt werden, wenn entweder alle Rentiere aus dem Urlaub zurückgekehrt und bereit für die Auslieferung am 24.12. sind, oder mindestens eine bestimmte Menge (p) von Elfen ein Problem bei der Geschenkproduktion haben und Santas Hilfe benötigen.
- Die **Rentiere** haben dabei Priorität und werden am 24.12. vor den Elfen bedient.
- Wenn p **Elfen** ein Problem haben, kümmert der Weihnachtsmann sich um alle p gleichzeitig.

Zusätzlich zu diesen klassischen Anforderungen habe ich weitere Annahmen getroffen, um die Ausgabe nachvollziehbarer zu gestalten. Die Simulation funktioniert zeitgesteuert, wobei ein Tag im Jahreslauf der Simulation einer Sekunde Echtzeit entspricht.

Die Rückkehr der Rentiere sowie die Problemquote der Elfen basieren auf stochastischen Modellen, was die Komplexität der Synchronisation minimal erhöht, aber einen sinnvolleren und Nachvollziehbareren Log liefert, bei dem nicht alle Rentiere zeitgleich eintreffen.

2. Aufgabe 1 - Semaphoren Ansatz

In der ersten Aufgabe wird das Problem lokal mit Semaphoren in Python implementiert. Die Semaphoren werden dabei verwendet um den Zugriff auf geteilte Variablen zu steuern.

2.1 Synchronisation

In dieser Implementierung gibt es vier solcher Variablen:

- *santa_sem* (initial 0) stellt den wartenden Weihnachtsmann dar. Er wird mittels *release()* geweckt, sobald alle Rentiere da sind oder p Elfen Hilfe benötigen
- *reindeer_sem* (initial 0) ist eine gemeinsame Variable der Rentiere und zählt wie viele von ihnen bereit sind. Mit *acquire()* melden sich die Zugtiere bei ihrer Ankunft am Nordpol bereit und blockieren, bis sie nach erfolgreicher Geschenkeauslieferung von Santa gemeinsam wieder frei gegeben werden.

- *elf_sem* (initial 0) ist analog der Zähler für die Elfen. Ein Elf der auf ein Problem trifft meldet dieses hier. Wenn der Wert auf p behandelt Santa die Probleme, indem er die Prozesse wieder frei gibt.
- *mutex* (initial 1) steuert den Schreibzugriff auf die geteilten Variablen.

```
37  def santa():
38      global elf_count, reindeer_count
39      log(CLR_SANTA, "Santa", "Entspannung im Hauptquartier.")
40  while simulation_running:
41      santa_sem.acquire()
42
43      with mutex:
44          if reindeer_count == NUM_REINDEER:
45              log(CLR_SANTA, "Santa", "Ho Ho Ho! Alle Rentiere sind da. Abflug!")
46              reindeer_count = 0
47              for _ in range(NUM_REINDEER):
48                  reindeer_sem.release()
49              time.sleep(0.1)
50
51          elif elf_count >= ELVES_NEEDED:
52              log(CLR_SANTA, "Santa", f"Hilft einer Gruppe von {ELVES_NEEDED} Elfen.")
53              for _ in range(ELVES_NEEDED):
54                  elf_sem.release()
55              elf_count -= ELVES_NEEDED
```

Abb. 1 Verwendung von Semaphoren in der Santa-Funktion

2.2 Zeitsteuerung

Da sich alle Threads denselben Adressraum teilen, kann das Datum über eine globale Variable gesteuert werden. Ein Taktgeber-Thread erhöhte in dieser Variable jede Sekunde das Datum. Obwohl es sich um eine geteilte Variable handelt, ist hier kein Semaphor nötig, da nur der Taktgeber schreibend auf die Variable zugreift, während die restlichen Threads ausschließlich aus ihr lesen.

2.3 Fehleranfälligkeit und Rentierrückkehr

Zwischen dem 24.12. und 01.01. sind am Nordpol Betriebsferien, an den restlichen Tagen des Jahrs überprüft jeder Elf-Thread am Beginn eines Arbeitstags ob er auf ein Problem trifft, dies geschieht mit der konfigurierbaren Wahrscheinlichkeit p_e (standardmäßig 5%).

Die Rentiere treffen zwischen dem 16.12 und 24.12 mit zunehmender Wahrscheinlichkeit ein. Wenn nichts schief läuft und an Heiligabend alle vor Ort sind, startet der Weihnachtsmann die Geschenke Auslieferung.

3. Aufgabe 2 - Docker und ZeroMQ

Schwerpunkt des Aufgabenblatts ist die zweite Implementation mit Docker und ZeroMQ, die alle Akteure des weihnachtlichen Experiments als eigene Container implementiert

3.1 Bekannte Struktur

Wo vorher drei Klassen von Threads existiert haben (den Zeitgeber ausgenommen), gibt es nun analog drei Typen von Containern.

1. **Santa:** Der Weihnachtsmann übernimmt zusätzlich zu seiner Rolle als Koordinator nun auch die Aufgabe des Taktgebers.
2. **Elfen:** Die fleißigen Arbeiter übernehmen die selben Aufgaben wie zuvor, mit Betriebsferien zwischen den Jahren und zufälliger Fehlersimulation an den restlichen Tagen.
3. **Rentier:** Auch die Zugtiere funktionieren genau wie bisher und treffen zwischen dem 16. und 24. Dezember zur Geschenkeauslieferung ein. Um die rechtzeitige Ankunft sicherzustellen, ist eine exponentielle Wahrscheinlichkeitsfunktion implementiert:

$$P(d) = \frac{2^{d-16}}{100} \text{ für } 16 \leq d \leq 24$$

So ist die Wahrscheinlichkeit am 16. Dezember bei 1 % und verdoppelt sich täglich, sodass spätestens am 23. Dezember mit 128 % Eintreffwahrscheinlichkeit alle Rentiere versammelt sind. Bis zum 31. des Monats brechen sie unter Verwendung der selben Wahrscheinlichkeitsfunktion wieder in den Urlaub im Süden auf.

3.2 Message Passing

Statt geteilter Variablen wird in diesem Ansatz auf Nachrichtenaustausch umgestellt. Die Bibliothek ZeroMQ setzt diesen als Middleware um.

Es werden zwei Kommunikationsmuster verwendet:

- A. Publish/Subscribe:** Der Main-Container (Santa) verwende einen PUB-Socket um regelmäßig das aktuelle Datum zu kommunizieren (TIME:2025-12-24). Alle anderen Container sind diesem über einen SUB-Socket subscribed und empfangen die gesendeten Nachrichten synchron. Neben der Uhrzeit wird über dieses System auch der Aufbruch in der Weihnachtsnacht publiziert (EVENT:FLY_AWAY).
- B. Push/Pull:** Elfen und Rentiere nutzen zur Mitteilung von Statusänderungen Push-Mitteilungen an den PULL-Socket des Weihnachtsmanns. Der PULL-Socket ist mit FIFO gequeued, Santa arbeitet die Eingehenden Mitteilung chronologisch ab.

3.3 Deployment

Die Verwaltung der Container erfolgt über Docker Compose (*docker-compose.yml*), während die Simulationsparameter in einer Pythondatei (*app/config.py*) konfiguriert sind. Die Verwendung von Kubernetes ist in diesem Fall nicht wirklich sinnvoll, da das Netzwerk nur lokal laufen soll und von überschaubarer Größe ist.

Konkret werden drei verschiedene Container erzeugt (santa, elf und reindeer) und instanziiert. Mittels Umgebungsvariablen werden die Reintiercontainer mit dem jeweiligen Namen parametrisiert erzeugt. Die Kommunikation erfolgt im Anschluss über ein isoliertes Bridge-Netzwerk.

4. Logging

In beiden Varianten der Umsetzung wird der Log direkt in Python farblich kodiert, um eine Unterscheidung zwischen den Akteurtypen zu vereinfachen. Jedem Log ist das Datum innerhalb der Simulation vorangestellt.

santa	[11.12.] [Santa]: Hlft Gruppe: ['e5fc', 'fde3', '2f6e', 'e291', '24d2']
elf	[11.12.] [Elf e291]: Problem gelöst! [11.12.] [Elf e5fc]: Problem gelöst! [11.12.] [Elf 24d2]: Problem gelöst! [11.12.] [Elf fde3]: Problem gelöst! [11.12.] [Elf 2f6e]: Problem gelöst! [12.12.] [Elf 93b4]: Problem! [19.12.] [Elf 24d2]: Problem! [20.12.] [Elf c645]: Problem!
blitzen	[21.12.] [Blitzen]: Zurück am Nordpol!
vixen	[21.12.] [Vixen]: Zurück am Nordpol!
donner	[22.12.] [Donner]: Zurück am Nordpol!
cupid	[22.12.] [Cupid]: Zurück am Nordpol!
rudolph	[22.12.] [Rudolph]: Zurück am Nordpol!
dancer	[22.12.] [Dancer]: Zurück am Nordpol!
comet	[23.12.] [Comet]: Zurück am Nordpol!
dasher	[23.12.] [Dasher]: Zurück am Nordpol!
prancer	[23.12.] [Prancer]: Zurück am Nordpol!
santa	[24.12.] [Santa]: Ho Ho Ho! Alle Rentiere da. Abflug!

Abb.2 Farblich codierter Log. Screenshot aus Docker Desktop.

5. Fazit

Das Santa Claus Problem eignet sich gut um die Unterschiede beider Architekturparadigmen zu betrachten. Während Aufgabe 1 die klassischere Welt der Betriebssystem-Programmierung auf einem einzelnen Host repräsentiert, spiegelt Aufgabe 2 die Herausforderungen beziehungsweise Möglichkeiten moderner cloudnativer Architekturen wieder.

Der fundamentale Unterschied zwischen beiden Ansätzen findet sich im Übergang von Shared Memory zu Message Passing.

In Aufgabe 1 ist die Synchronisation aller Prozesse implizit durch den gemeinsamen Speicher gegeben. Durch das Teilen müssen dessen kritischen Abschnitte aber auch durch Locks, wie Semaphoren, geschützt werden. Vorteil ist die extrem geringe Latenz und die einfache Handhabung globaler Zustände (z.B. Datum der Simulation). Nachteilig ist die mangelnde Skalierbarkeit über mehr als ein Gerät sowie die immer bestehende Gefahr von Deadlocks, wenn Semaphore verwendet und möglicherweise nicht ordnungsgemäß implementiert sind.

Aufgabe 2 setzt an den Nachteilen der vorherigen Variation an und behebt diese. Durch die Isolation der Container können diese auf verschiedenen Systemen laufen, dafür fällt der geteilte

Speicher weg. Alle Synchronisationssignale müssen explizit als Nachrichten formuliert werden was die Komplexität des Programms massiv erhöht. Die erreichte Steigerung der Robustheit durch vollständige Verhinderung von „Race Conditions“ auf gemeinsamen Variablen wird durch eine neuen Fehlerquelle, das Netzwerk, aufgewogen.

Die Containerisierung mittels Docker und die Orchestrierung durch Docker Compose zeigt die Vorteile der horizontalen Skalierbarkeit gut auf. Das System startete überraschend problemlos nach dem die einzige Fehlermeldung darauf zurückzuführen war, dass die Docker Desktop-App noch nicht lief.

Dank der gewählten Architektur könnte der Weihnachtsmann außerdem Problemlos weitere Elfen einstellen, die Anzahl ist mit einer einzigen Variable in der docker-compose.yml anpassbar.

Ein weitere Vorteil ist die erhöhte Fehlertoleranz. Sollte ein Elf-Container unerwartet abstürzen, hat das keine Auswirkungen auf den Weihnachtsmann oder seine Rentiere. Im Semaphor-Modell hingegen könnte ein unbehandelter Fehler das gesamte System zum Absturz bringen.